



T.C.

**İSTANBUL BEYKENT ÜNİVERSİTESİ
MÜHENDİSLİK-MİMARLIK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ (TR)**

**BİLGİSAYAR MÜHENDİSLİĞİ TASARIM PROJESİ
AI Tabanlı Araç Fiyat Değerleme ve Karar Destek Asistanı**

Projeyi Hazırlayan Öğrenciler

Elman MAREFAT 2203022120

Görkem Furkan ÇAĞLAYAN 2203022068

Danışman

Dr. Öğr. Üyesi Talat FİRLAR

1. Frontend Kod Yapısının Genel Mantığı

Bu arayüz, kullanıcıya araç fiyat analizi ve karar destek çıktıları sunmak amacıyla **bileşen tabanlı bir frontend mimarisi** ile geliştirilmiştir. Kod yapısında React ve TypeScript kullanılmış, her ekran veya panel ayrı bir bileşen olarak tasarlanmıştır. Bu yaklaşım sayesinde hem kod okunabilirliği artmış hem de sistemin ileride genişletilebilir olması sağlanmıştır.

Arayüz temel olarak iki ana sayfadan oluşmaktadır:

- Ana Sayfa (HomePage)
- Sonuç Paneli / Dashboard

Bu iki sayfa arasındaki geçiş, App.tsx dosyasında tutulan görünüm durumu (view state) ile kontrol edilmektedir.

2. App.tsx Dosyası – Uygulamanın Ana Kontrol Katmanı

Ekran görüntüsünde görülen kod parçası şu mantıkla çalışmaktadır:

```
import { useState } from 'react';
import { HomePage } from '@app/components/HomePage';
import { Dashboard } from '@app/components/Dashboard';

export default function App() {
  const [currentView, setCurrentView] = useState<'home' | 'dashboard'>('home');

  return (
    <div className="font-sans">
      {currentView === 'home' ? (
        <HomePage onNavigateToDashboard={() =>
          setCurrentView('dashboard')} />
      ) : (
        <Dashboard onNavigateToHome={() => setCurrentView('home')} />
      )}
    </div>
  );
}
```

Bu kod ne yapıyor?

Bu dosya, uygulamanın ana giriş noktasıdır. Uygulama açıldığında hangi ekranın gösterileceğine burada karar verilir. `useState` kullanılarak `currentView` adında bir durum değişkeni tanımlanmıştır. Bu değişkenin değeri başlangıçta "home" olarak belirlenmiştir. Bu nedenle uygulama ilk açıldığında kullanıcı ana ekranı görmektedir.

Kullanıcı analiz işlemini başlattığında `setCurrentView('dashboard')` çağrılır ve ekran otomatik olarak Dashboard sayfasına geçer. Benzer şekilde Dashboard ekranında ana sayfaya dönmek için `setCurrentView('home')` kullanılır.

Neden bu yapı seçilmiş?

Bu yapı küçük ve orta ölçekli prototipler için oldukça uygundur. Çünkü bu aşamada tam sayfa yönlendirmesi yapan bir router sistemi kurmadan, iki ana görünüm arasında sade bir geçiş mekanizması sağlanmıştır. Bu da projeyi hem daha anlaşılır hem de daha hızlı geliştirilebilir hale getirmiştir.

3. HomePage.tsx – Ana Ekran Bileşeni

Ana ekran bileşeni, kullanıcının sistemle ilk etkileşime geçtiği bölümdür. Ekran görüntülerinden anlaşıldığı üzere bu bölümde şu öğeler bulunmaktadır:

- Proje başlığı
- Alt açıklama metni
- URL veya araç bilgisi girişi için bir alan
- “Analyze Listing URL” / “Manual Vehicle Entry” butonları
- Alt bölümde sistemin temel yeteneklerini gösteren ikonlu kartlar

Bu bileşenin görevi nedir?

HomePage.tsx, kullanıcıdan ilk girdiyi almak ve kullanıcıyı analiz sürecine yönlendirmek için kullanılır. Arayüzün sade tutulmasının nedeni, kullanıcı deneyimini kolaylaştırmak ve sistemin ilk izlenimini karmaşık olmayan bir yapıda sunmaktır.

4. Dashboard.tsx – Sonuçların Sunulduğu Ana Panel

Dashboard bileşeni, analiz sonucunda elde edilen tüm verilerin kullanıcıya gösterildiği ana ekrandır. Görsellerden anlaşıldığı üzere bu sayfada:

- Fiyat aralığı paneli
- Price gauge
- Araç bilgi kartı
- Yapay zeka değerlendirme kartı
- Risk skoru kartı
- Karar destek analizi tablosu

yer almaktadır.

Bu bileşen ne yapar?

Dashboard, sistemin farklı alt bileşenlerinden gelen çıktıları tek bir ekranda toplar. Bu sayede kullanıcı hem fiyat tahminini hem de risk analizini aynı anda görebilir. Teknik olarak bu dosya çoğunlukla diğer alt bileşenleri çağıran bir üst seviye bileşendir.

5. Bileşen Tabanlı Yapının Önemi

Kod yapısında her panelin ayrı bileşenlere ayrılmış olması çok önemli bir mühendislik tercihidir. Örneğin:

- PriceGauge.tsx → fiyat barı

- AISummaryCard.tsx → LLM/NLP çıktısı
- RiskScoreCard.tsx → risk göstergesi
- DecisionTable.tsx → karar destek tablosu
- VehicleInfoCard.tsx → araç özet bilgisi

Bu sayede sistem tek bir büyük dosya yerine küçük ve yönetilebilir modüller halinde tasarlanmıştır.

Neden bu iyi bir şey?

- Kod tekrarını azaltır
 - Hata ayıklamayı kolaylaştırır
 - Görsel düzeni değiştirmeyi kolaylaştırır
 - Her bileşen daha sonra gerçek veri ile ayrı ayrı beslenebilir
-

6. Arayüz Bileşenlerinin Kod Yapısı ve İşlevleri

Frontend prototipinde kullanılan alt bileşenler, kullanıcıya farklı analiz sonuçlarını düzenli ve anlaşılır şekilde sunmak amacıyla ayrı modüller halinde tasarlanmıştır. Bu yaklaşım sayesinde arayüz hem daha okunabilir hem de teknik olarak daha sürdürülebilir bir yapıya kavuşmuştur.

6.1 PriceGauge.tsx – Fiyat Metresi Bileşeni

Bu bileşen, sistemin tahmin ettiği fiyat aralığını ve mevcut ilan fiyatını tek bir görsel yapı üzerinde göstermektedir. Ekran görüntülerinde görülen yeşil–sarı–kırmızı skala, kullanıcının analiz edilen aracın piyasa içerisindeki konumunu hızlıca anlayabilmesi amacıyla tasarlanmıştır.

Bu bileşen ne yapar?

PriceGauge.tsx bileşeni, minimum ve maksimum tahmin değerleri ile mevcut ilan fiyatını bir araya getirerek bir fiyat skoru görselleştirmesi oluşturur. Kullanıcı bu panel üzerinden aracın:

- fırsat seviyesinde olup olmadığını,
- piyasa aralığında mı bulunduğunu,

- yoksa olması gerekenden pahalı mı olduğunu tek bakışta anlayabilmektedir.

Neden bu yapı kullanılmıştır?

Karar destek sistemlerinde yalnızca sayısal bir fiyat göstermek çoğu zaman yeterli olmamaktadır. Fiyatın “anlamlandırılması” gerekir. Bu nedenle çizgisel veya bar biçiminde bir görselleştirme kullanılarak kullanıcının karar verme süreci hızlandırılmıştır.

Teknik olarak nasıl çalışır?

Bu bileşen genellikle şu verileri alır:

- Tahmini minimum fiyat
- Tahmini maksimum fiyat
- Mevcut ilan fiyatı

Ardından bu fiyatlar normalize edilerek, ilan fiyatının skala üzerindeki pozisyonu hesaplanır. Böylece dinamik bir ibre veya konumlandırma yapılır..

6.2 AISummaryCard.tsx – Yapay Zeka Değerlendirme Kartı

Bu bileşen, doğal dil işleme ve büyük dil modeli destekli analizlerin kullanıcıya sunulduğu en önemli bölümlerden biridir. Ekran görüntüsünde “Painted parts detected” ve “Previous accident record found” gibi risk ifadeleri bu kart üzerinde gösterilmektedir.

Bu bileşen ne yapar?

AISummaryCard.tsx, ilan açıklamasından veya sistemin analiz çıktılarından elde edilen metinsel bilgileri kullanıcıya anlaşılır bir biçimde sunar. Bu kartta:

- riskli ifadeler,
- olumlu ifadeler,
- kısa yapay zeka yorumu yer alır.

Neden önemli?

Bu proje yalnızca fiyat tahmini yapan bir sistem değildir; aynı zamanda bir **karar destek asistanıdır**. Bu nedenle kullanıcıya yalnızca fiyat vermek yerine, neden o sonuca ulaşıldığını açıklayan bir yorum alanı sunmak gerekir. Bu kart, sistemin “açıklanabilir yapay zeka” boyutunu temsil etmektedir.

Teknik mantık

Bu kart ileride şu kaynaklardan beslenebilir:

- NLP tabanlı anahtar ifade tespiti
- LLM özetleme çıktısı
- yapılandırılmış risk sınıflandırmaları

Şu anki prototip sürümünde ise bu bilgiler mock/fake veri ile gösterilmekte ve arayüz yapısı test edilmektedir.

6.3 RiskScoreCard.tsx – Risk Skoru Kartı

Bu bileşen, aracın genel risk seviyesini hem sayısal hem de görsel olarak kullanıcıya sunmaktadır. Görselde risk skoru 62 olarak ve ayrıca teknik risk, fiyat riski ve piyasa riski gibi alt başlıklara ayrılmış şekilde gösterilmektedir.

Bu bileşen ne yapar?

RiskScoreCard.tsx, farklı risk bileşenlerini bir araya getirerek genel bir risk skoru oluşturur ve bunu kullanıcıya sade bir kart yapısı içinde sunar.

Neden böyle tasarlanmıştır?

İkinci el araç satın alma kararında sadece fiyat yeterli değildir. Araç, fiyat açısından uygun görünse bile:

- teknik risk taşıyabilir,
- geçmiş hasar barındırabilir,
- piyasa açısından riskli olabilir.

Bu nedenle risk skoru ayrı bir panel olarak tasarlanmıştır. Böylece fiyat ve risk birbirinden ayrıştırılarak kullanıcıya daha net bir karar desteği sunulmaktadır.

Teknik yapı:

Bu kart şu değerleri tutabilir:

- toplam risk puanı
- teknik risk yüzdesi
- fiyat riski yüzdesi
- piyasa riski yüzdesi

Bunlar gelecekte model veya kurallar tabanı ile üretilebilir.

6.4 DecisionTable.tsx – Karar Destek Analizi Tablosu

Bu bileşen, projenin en güçlü mühendislik unsurlarından biridir. Çünkü sistemin verdiği sonucu hangi faktörlere dayandırıldığını şeffaf biçimde göstermektedir.

Bu bileşen ne yapar?

DecisionTable.tsx, fiyat tahmini sürecinde etkili olan faktörleri tablo halinde sunar. Her satırda:

- özellik adı,
- analiz sonucu,
- fiyata etkisi,
- veri kaynağı

bulunmaktadır.

Örnek:

- Kilometre → Ortalama üstü → -%5
- Hasar kaydı → 15.000 TL → -%3
- Boya/değişen → Kaput → -%10

Neden bu kadar önemli?

Bu tablo, projeyi “sadece tahmin yapan bir model” olmaktan çıkarıp

açıklanabilir karar destek sistemi haline getirir. Çünkü kullanıcı yalnızca sonucu değil, sonuca hangi etkenlerin ne oranda katkıda bulunduğunu da görmektedir.

Teknik katkısı:

Bu bileşen ileride:

- scraping çıktıları,
- NLP bulguları,
- fiyat modeli sonuçları

ile otomatik olarak doldurulabilir. Şimdiki sürümde ise arayüz prototipinin mantığını göstermek amacıyla örnek veri ile çalışmaktadır.

6.5 VehicleInfoCard.tsx – Araç Bilgileri Kartı

Bu kart, analiz edilen araca ait temel özellikleri özet biçimde kullanıcıya gösterir. Görselde marka-model, yıl, kilometre gibi bilgiler bu kartta yer almaktadır.

Bu bileşen ne yapar?

VehicleInfoCard.tsx, fiyat tahmini ve risk analizi için kullanılan temel giriş verilerini özetler. Böylece kullanıcı sistemin hangi verilerle çalıştığını görebilir.

Neden ayrı bir kart olarak verildi?

Analiz sonuçlarının güvenilir görünmesi için giriş verilerinin kullanıcı tarafından açıkça görülebilmesi gerekir. Kullanıcı, sistemin hangi aracı analiz ettiğini bu kart sayesinde doğrulayabilir.

6.6 Bu Bileşenlerin Birlikte Çalışma Mantığı

Bu bileşenler birbirinden bağımsız gibi görünse de aslında aynı analiz akışının farklı çıktılarıdır. Dashboard ekranında:

- VehicleInfoCard → giriş verilerini özetler
- PriceGauge → fiyat aralığını görselleştirir
- AISummaryCard → metinsel değerlendirme sunar
- RiskScoreCard → risk düzeyini gösterir
- DecisionTable → karar destek detaylarını açıklar

Bu yapı sayesinde kullanıcıya hem üst düzey hem de detaylı sonuçlar sunulmaktadır.

7. Proje Klasör Yapısı

Ekran görüntülerine göre proje yapısı şu şekilde:

src/

```
|— app/
|   |— components/
|   |   |— HomePage.tsx
|   |   |— Dashboard.tsx
|   |   |— PriceGauge.tsx
|   |   |— AISummaryCard.tsx
|   |   |— RiskScoreCard.tsx
|   |   |— DecisionTable.tsx
|   |   |— VehicleInfoCard.tsx
|   |
|   |— App.tsx
|
|— styles/
|   |— tailwind.css
|   |— index.css
|
```

README.md

package.json

Bu yapı ne anlama geliyor?

Bu proje, modern web geliştirme standartlarına uygun şekilde **katmanlı ve modüler bir yapı** ile geliştirilmiştir.

- components/ → Arayüz parçaları
- App.tsx → uygulama kontrolü
- styles/ → görsel tasarım
- README.md → proje açıklaması

8. Kullanılan Teknolojiler (Frontend)

Bu proje arayüzü geliştirilirken modern web teknolojileri kullanılmıştır.

Kullanılan başlıca teknolojiler:

- **React** → bileşen tabanlı UI geliştirme
- **TypeScript** → tip güvenliği
- **Tailwind CSS** → hızlı stil verme
- **Figma** → arayüz tasarım

9. Backend Kod Yapısının Genel Mantığı

Bu bölüm, projenin makine öğrenmesi tabanlı arka uç sisteminin tasarım kararlarını, kullanılan teknolojiyi ve temel çalışma prensiplerini açıklamaktadır. Backend sistemi; veri yönetimi, model eğitimi ve tahmin üretimi olmak üzere üç temel katmandan oluşmaktadır.

Sistemin genel veri akışı şu şekilde işlemektedir: Kullanıcı araç bilgilerini girer, bu bilgiler özellik mühendisliği aşamasından geçer, ardından eğitilmiş model tahmini üretir ve sonuçlar kullanıcıya sunulur.

10. Kullanılan Teknolojiler (Backend)

Backend sistemin geliştirilmesinde sektör standardı kabul edilen Python ekosistemi tercih edilmiştir. Bu tercih; kütüphane zenginliği, okunabilirlik ve makine öğrenmesi alanındaki yaygın kullanımı nedeniyle yapılmıştır.

Teknoloji	Kullanım Amacı
Python 3.x	Ana geliştirme dili
pandas	CSV okuma, veri manipülasyonu ve ön işleme
scikit-learn	Makine öğrenmesi model eğitimi ve değerlendirme
RandomForestRegressor	Fiyat tahmini için seçilen algoritma
LabelEncoder	Kategorik verilerin sayısal temsile dönüştürülmesi
Virtual Environment (venv)	Bağımlılık izolasyonu

11. Veri Seti Yapısı ve Yönetimi

11.1 CSV Yapısı

Sistem, yapılandırılmış bir CSV dosyası üzerinde çalışmaktadır. Her satır bir araç ilanını temsil etmekte olup aşağıdaki sütunlardan oluşmaktadır:

Sütun Adı	Veri Tipi	Açıklama
marka	Kategorik	Araç markası (Ford, Toyota, BMW vb.)
model	Kategorik	Araç modeli (Focus, Corolla, 3 Serisi vb.)
yil	Sayısal (int)	Araç üretim yılı
km	Sayısal (int)	Araç kilometresi
yakit	Kategorik	Yakıt tipi (Benzin, Dizel, Hibrit, LPG, Elektrik)
vites	Kategorik	Vites tipi (Manuel, Otomatik)
fiyat	Sayısal (int)	Hedef değişken - araç satış fiyatı (TL)
kaynak	Kategorik	Verinin kaynağı (sentetik / gerçek)

11.2 Veri Toplama Süreci ve Aşamalı Yaklaşım

Proje, bilinçli olarak aşamalı bir veri toplama stratejisi benimsemiştir. Bu yaklaşımın temel gerekçesi, modelin davranışını küçük ve kontrollü veri setleri üzerinde gözlemleyerek kademeli iyileştirme yapmaktır.

- 1. Aşama — Sentetik Veri: Piyasa gözlemine dayalı olarak araştırmacı tarafından manuel derlenen başlangıç veri seti. Modelin temel davranışını test etmek amacıyla kullanılmıştır.
- 2. Aşama — Gerçek Veri Entegrasyonu: arabam.com gibi platformlardan manuel olarak toplanan gerçek ilan verileri sisteme eklenmiştir. Her veri satırı 'kaynak' sütunu ile etiketlenmektedir.
- 3. Aşama (Planlanan) — Web Scraping: Veri toplama süreci otomatize edilerek gerçek zamanlı ilan verilerine erişim sağlanacaktır.

'kaynak' sütunu bu geçiş sürecini yönetmek için kritik öneme sahiptir. Sentetik veriler bu sütun üzerinden filtrelenerek sistem gerçek veriye tam geçiş yaptığında kolayca çıkarılabilecektir.

12. Model Mimarisi

12.1 Algoritma Seçimi: RandomForestRegressor

Fiyat tahmini için Random Forest algoritması tercih edilmiştir. Bu tercih aşağıdaki teknik gerekçelere dayanmaktadır:

- Kategorik ve sayısal özelliklerin bir arada bulunduğu veri setlerinde yüksek performans göstermesi
- Aşırı öğrenmeye (overfitting) karşı doğal direnci
- Az veriyle de makul tahminler üretebilmesi
- Özellik önem sıralaması (feature importance) çıktısı sayesinde yorumlanabilirlik sunması

12.2 Veri Ön işleme

Makine öğrenmesi modelleri yalnızca sayısal verilerle çalışabildiğinden, kategorik sütunlar eğitim öncesinde sayısal temsile dönüştürülmüştür. Bu işlem scikit-learn kütüphanesinin LabelEncoder sınıfı ile gerçekleştirilmektedir.

```
encoders = {}
for sutun in ['marka', 'model', 'yakit', 'vites']:
    le = LabelEncoder()
    df[sutun] = le.fit_transform(df[sutun])
    encoders[sutun] = le
```

Her encoder ayrı saklanmaktadır. Bu yaklaşım, kullanıcı giriş verisinin eğitim aşamasında kullanılan encoder ile tutarlı biçimde dönüştürülmesini garanti altına alır.

12.3 Eğitim/Test Veri Bölümü

Model performansının güvenilir biçimde ölçülebilmesi için veri seti eğitim ve test kümelerine ayrılmaktadır.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- test_size=0.2: Verinin %80'i eğitim, %20'si test için ayrılmaktadır.
- random_state=42: Rastgele bölümlleme sabitlenmektedir. Bu parametre, farklı çalıştırmalarda aynı bölümllemenin oluşmasını sağlayarak sonuçların karşılaştırılabilirliğini korur.

Eğitim verisi modelin öğrenmesi için kullanılırken, test verisi modelin hiç görmediği örneklerle değerlendirilmesi için ayrılmaktadır. Bu ayrımın temel amacı ezber (overfitting) durumunu önlemektir.

13. Model Performansı ve Aşamalı İyileşme

Projenin en önemli akademik katkılarından biri, model performansının veri miktarı arttıkça sistematik olarak ölçülmesidir. Her geliştirme aşamasında ortalama mutlak hata (Mean Absolute Error - MAE) hesaplanarak kayıt altına alınmaktadır.

Aşama	Veri Miktarı	Ortalama Hata Payı	Değişim
Aşama 1	20 satır (sentetik)	124.500 TL	Başlangıç noktası
Aşama 2	101 satır (sentetik)	115.943 TL	-8.557 TL
Aşama 3	201 satır (karma)	326.336 TL	Sentetik/gerçek çakışması

Aşama 3'te hata payının belirgin biçimde artması, sentetik ve gerçek veriler arasındaki fiyat seviyesi farkından kaynaklanmaktadır. Sentetik veriler 2022-2023 fiyat seviyesinde üretilmişken, gerçek ilanlar 2026 Türkiye piyasasını yansıtmaktadır. Bu bulgu, veri kalitesinin veri miktarından daha belirleyici olduğunu açıkça ortaya koymaktadır.

Sonraki aşamada sentetik veriler kademeli olarak kaldırılacak ve yalnızca gerçek ilan verileriyle model yeniden değerlendirilecektir.

14. Terminal Arayüzü (model.py)

Mevcut prototip aşamasında sistem, web arayüzüne bağlanmadan önce terminal üzerinden iki modda çalışmaktadır. Bu tasarım kararı, modelin doğruluğunu arayüz bağımlılığı olmadan test edebilmek amacıyla alınmıştır.

14.1 Mod 1 — Otomatik Test

Veri setinin %20'si rastgele test verisi olarak ayrılır ve model bu verileri tahmin eder. Her araç için gerçek fiyat, tahmin edilen fiyat ve fark gösterilir. Ortalama hata payı hesaplanarak ekrana yazdırılır.

14.2 Mod 2 — Manuel Araç Girişi

Kullanıcı bir araç için marka, model, yıl, kilometre, yakıt tipi ve vites bilgilerini girer. Sistem bu bilgileri encode ederek tahmin üretir. Eğer girilen marka veya model veri setinde mevcut değilse, kullanıcıya uyarı mesajı gösterilir. Ek olarak, veri setindeki benzer araçlar referans olarak listelenir.

Bu arayüz, proje ortağının geliştirdiği React/TypeScript tabanlı frontend ile ilerleyen aşamada Flask API üzerinden entegre edilecektir.

15. Proje Klasör Yapısı

```
arac_asistan/  
├─ venv/                # Python sanal ortamı  
├─ arac_verisi.csv      # Ana veri seti  
├─ model.py             # Model eğitimi ve terminal arayüzü  
└─ veri_ekle.py         # Hızlı veri girişi yardımcısı
```

16. Planlanan Geliştirmeler

Aşağıdaki geliştirmeler sırasıyla hayata geçirilecektir:

#	Geliştirme	Açıklama
1	Sentetik Veri Tasfiyesi	Gerçek veri yeterli seviyeye ulaştığında sentetik veriler kaldırılacak, model yeniden eğitilerek hata payı karşılaştırması yapılacaktır.
2	Web Scraping Modülü	arabam.com ve benzeri platformlardan Selenium/BeautifulSoup ile otomatik veri toplama sistemi kurulacaktır.
3	Flask REST API	Model, HTTP endpoint'leri üzerinden frontend ile iletişim kuracak şekilde sarmalanacaktır.
4	LLM Entegrasyonu	İlan açıklama metinlerinin analizi için büyük dil modeli entegre edilecek; hasar tespiti, risk değerlendirmesi gibi çıktılar üretilecektir.
5	Model Kaydetme	Eğitilmiş model joblib ile diske kaydedilerek her istek için yeniden eğitim zorunluluğu ortadan kaldırılacaktır.

17. Temel Bulgular ve Çıkarımlar

Geliştirme süreci boyunca elde edilen teknik bulgular şu şekilde özetlenebilir:

- Model, otomatik ve manuel vites arasındaki fiyat farkını yalnızca veri üzerinden öğrenebilmektedir. Herhangi bir kural tanımlanmasına gerek yoktur.

- Veri kalitesi, veri miktarından daha belirleyici bir faktördür. Çakışan fiyat seviyeleri modeli olumsuz etkilemektedir.
- Veri setinde bulunmayan araç modelleri için tahmin güvenilirliği düşmektedir; sistem bu durumda kullanıcıyı uyarmaktadır.
- Veri setinin kilometre aralığı dışındaki değerler için tahmin sapması artmaktadır.

Bu bulgular, ilerleyen aşamalarda yapılacak iyileştirmeler için somut teknik gerekçe oluşturmaktadır ve final paper'ında sistematik biçimde raporlanacaktır.